Technical introduction to PSOC™ Edge E84 features

Chapter 4: Graphics

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 4 discusses PSOC™ Edge's Graphics features.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
2.1 Tools for graphics labs 4
3 Code example: Graphics using VGLite **5**
3.1 Objective 5
3.2 Description 5
3.3 Hardware setup 5
3.4 Flow chart 6
3.5 Project creation 7
3.6 Output 10
3.7 Modify the application 11
3.7.1 Render a different image 11
3.7.2 Draw polygons 14
3.8 Conclusion 16
4 Code example: Graphics LVGL demo **17**
4.1 Objective 17
4.2 Description 17
4.3 Flow chart 18
4.4 Project creation 18
4.5 Output 21
4.6 Modify the application 22
4.6.1 Pre-built LVGL demos 22
4.6.2 Custom UI using EEZ Studio 25
4.7 Conclusion 30
5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **31**
6 Appendix B: KIT_PSE84_EVAL details **34**
Revision history **35**
Disclaimer **36**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

2.1 Tools for graphics labs

Note that graphics labs utilize the following additional tools:

- EEZ-Studio v0.24.0 or later
  - This tool is used in Code example: Graphics LVGL demo. It is not required to run the default code example, but it's used to build a custom LVGL user interface (UI).
- Python 3
  - Including "pypng" and "lz4" Python modules:
    ```
    -python -m pip install pypng==0.20220715.0
    -python -m pip install lz4
    ```
  - Python 3 is not required to build the PSOC™ Edge code examples, but it is used by EEZ-Studio
- Pngquant v2.17.0 or later
  - Download binaries, unzip, and add folder to path environment variable
  - Pngquant is not required to build the PSOC™ Edge code examples, but it is used by EEZ-Studio

3 Code example: Graphics using VGLite

3.1 Objective

This code example demonstrates the utilization of Vivante® platform-independent VGLite graphics API to carry out hardware-accelerated 2D vector drawing operations and render the generated image on a 4.3” 800x480 display. The display is connected to the PSOC™ Edge E84 MCU via the MIPI Display Serial Interface (DSI). The code runs in a FreeRTOS environment.

3.2 Description

Resource initialization for this example is performed by this CM33 non-secure project. It configures the system clocks, pins, the clock to peripheral connections, and other platform resources. It then enables the CM55 core using the `Cy_SysEnableCM55()` function and allows the Idle task to put CM33 in DeepSleep mode.

In the CM55 application, the clocks and system resources are initialized by the BSP initialization function. The retarget-io middleware is configured to use the debug UART. The debug UART prints a message as shown in the Terminal output on program startup. The onboard KitProg3 acts as the USB-UART bridge to create the virtual COM port.

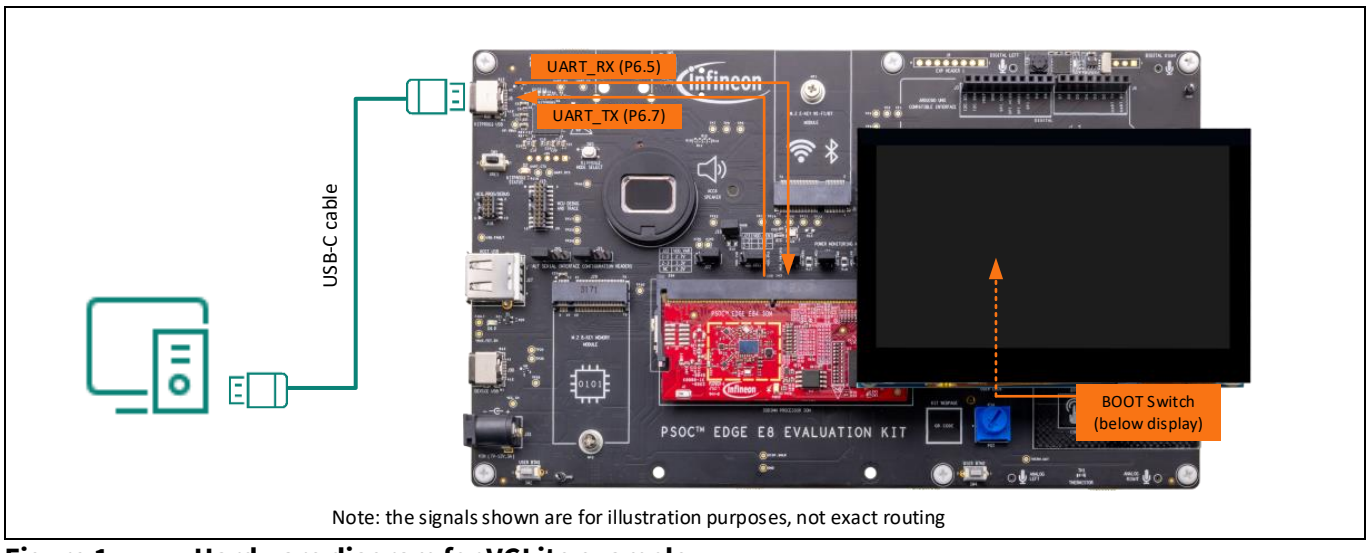
The CM55 application drives the LCD and renders the image using the PSOC™ Edge graphics subsystem, which houses an independent 2.5D GPU, a display controller (DC), and a MIPI DSI host controller with a MIPI D-PHY physical layer interface.

The GPU supports vector graphics (drawing circles, rectangles, quadratic curves) and font support. This code example implements operations like rotate/scale, color fill, and color conversion. After the GPU renders the frames, they are transferred to the MIPI DSI host controller via the DC and displayed on the LCD.

3.3 Hardware setup

This example uses the 4.3” Raspberry Pi DSI 800x480 display included with the PSOC™ Edge E84 EVK. To use a different display, please see the code example README.

1. Set BOOT SW in ON position.



Note: the signals shown are for illustration purposes, not exact routing

Figure 1 Hardware diagram for VGLite example

3.4 Flow chart

The following flow chart shows the complete code flow of this code example.

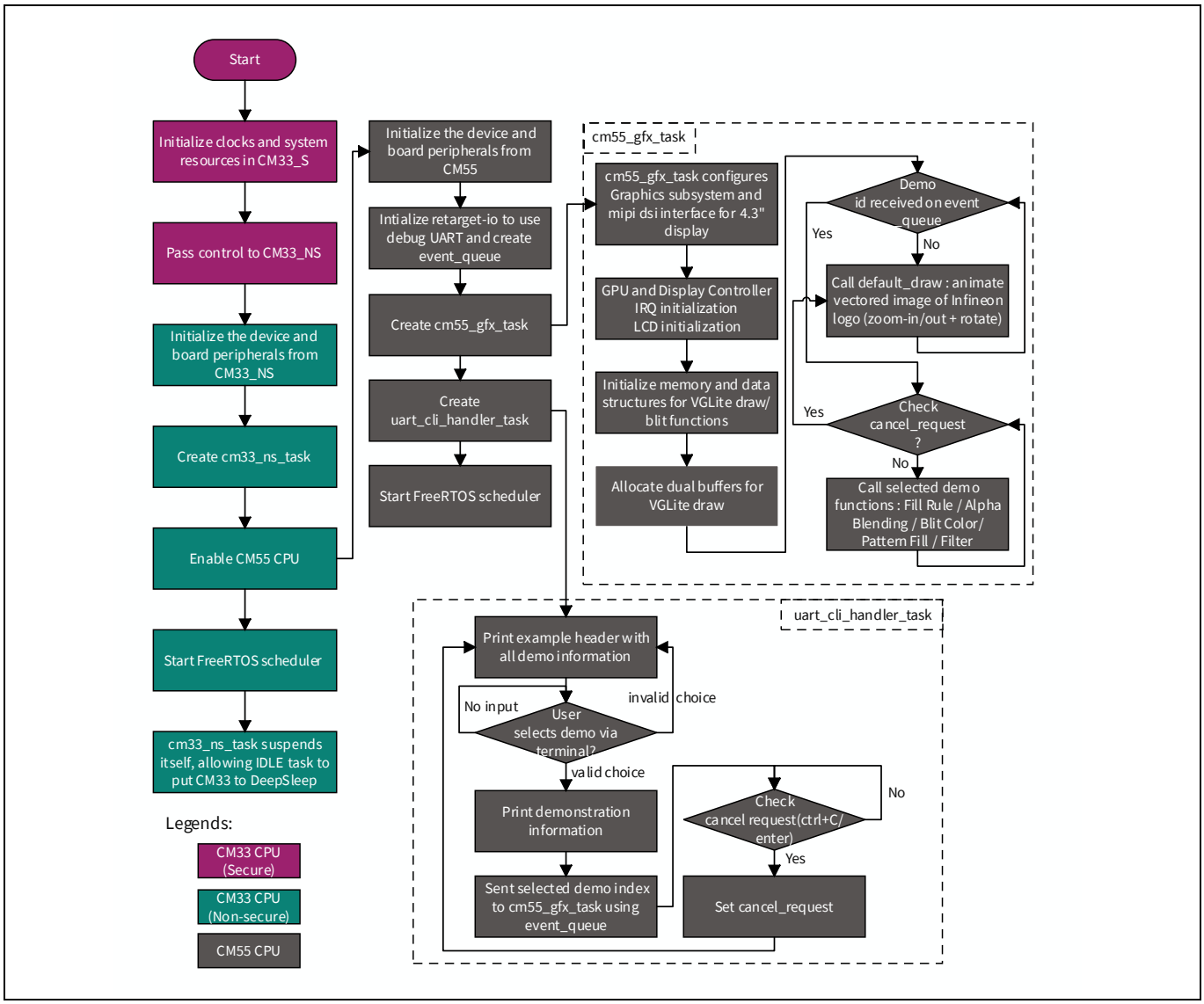


Figure 2 Flow diagram for graphics code example using VGLite

3.5 Project creation

2. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
3. When creating the application, select the **PSOC™ Edge Graphics using VGLite API** application under the **Graphics** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

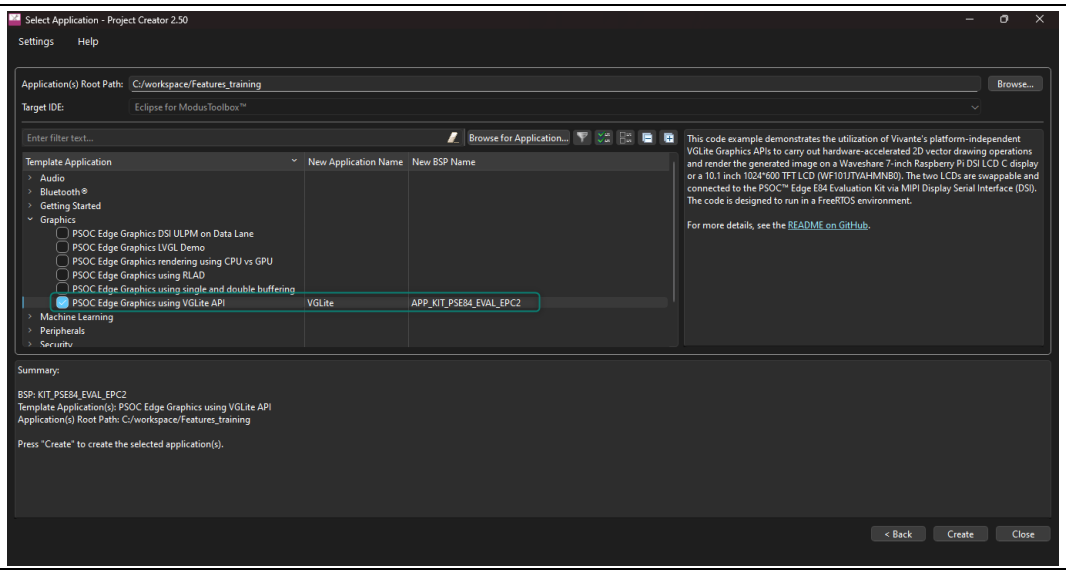


Figure 3 Creating VGLite example

4. Build and program the application.
5. **Note**: No modifications are needed to execute the default code example; however, some important files for this project are highlighted as shown in the following figure.

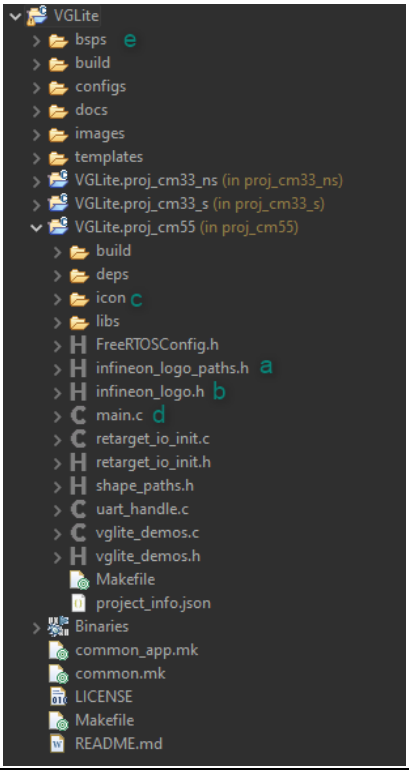


Figure 4 File structure for VGLite example

a) proj_cm55/infineon_logo_paths.h
This file defines the vector path data for the Infineon logo, which is rendered using the '`vg_lite_draw`' API. The path data is manually extracted from *images/infineon_logo.svg* by mapping the vector path opcodes required for plotting. This vector path is used in the '`default_draw`' function to animate the logo with zoom-in/out and rotation effects, and it is also utilized in the blit color rendering demo, where the rectangle containing the logo (rendered via '`vg_lite_draw`') is processed using the '`vg_lite_blit_rect`' API.

b) proj_cm55/infineon_logo.h
This file contains the C array holding the pixel data for the Infineon logo image. It is used in the Pattern Fill Demonstration to render the Infineon logo within four different shapes.

c) icon/*.h
These header files define the C arrays that hold pixel data for the four icons. They are used in the filter demo, where the VG_LITE_FILTER_LINEAR filter is applied to them.

d) Application code
The CM55 CPU utilizes the graphics subsystem and VGLite APIs to demonstrate five different use cases. The main function (*proj_cm55/main.c*) first initializes the BSP. It then performs retarget-io initialization to use the debug UART port and creates an event_queue to receive notifications of selected demonstrations. After this, it creates the cm55_gfx_task and uart_cli_handler FreeRTOS tasks.

i. **cm55_gfx_task**
This task is responsible for initializing the graphics subsystem. It initializes the LCD panel through the I2C interface, initializes the VGLite engine, creates buffers, and configures the identity matrix. After initialization, it enters a loop where it checks the '`event_queue`'. If no events are received, it calls the '`default_draw`' function to display the Infineon logo with zoom and rotation effects as the default screen. When a demo number is received, the corresponding demo function is executed repeatedly until the '`cancel_requested`' flag is set. Once cancellation is triggered, the task exits the demo loop and returns to the main loop, continuing to check the `event_queue` and call '`default_draw`'. It also logs FPS and CPU usage to the UART terminal throughout execution.

ii. **uart_cli_handler**
This task manages user interaction through the UART terminal. It first displays the header "PSOC™ Edge MCU: Graphics using VGLite API" and lists five GPU operations to be demonstrated. When the user selects one of the listed operations by entering its corresponding number (1 to 5), it prints information about the selected operation and sends the operation serial number to '`cm55_gfx_task`' via the '`event_queue`'. To exit a running GPU operation and return to the default display, the user can press Ctrl+C or Enter. When this occurs, the task sets the '`cancel_requested`' flag to notify '`cm55_gfx_task`' to stop the active operation and resume the default loop.

e) bsps/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/design.modus
This file stores Device Configurator information, display parameters, and memory configuration, among other configurations.

iii. To observe the graphics support configuration, open the **Device Configurator** tool from the **Quick Start** panel. From there, navigate to the **Peripherals** tab and check the **Graphics** option under **System** resource as shown below.

iv. Observe display parameters such as **Display Type**, **Width**, **Height**, **HSYNC**, and **VSYNC**. These parameters can be adjusted for other types of displays.

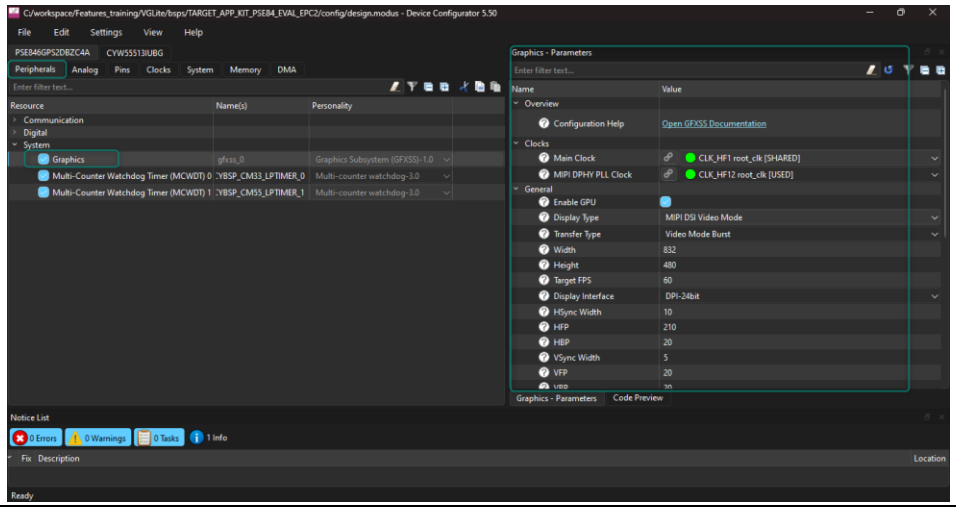


Figure 5 Display parameters in Device Configurator

v. Observe the reference clock for the MIPI DPHY PLL to 24 MHz.

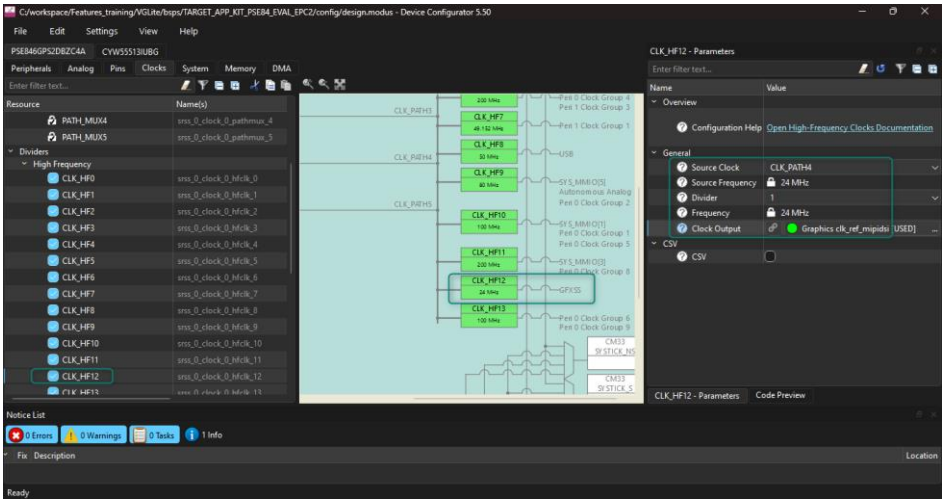


Figure 6 Clock configuration for MIPI DSI

vi. This file also defines the custom memory map for system SRAM with respect to the placement of the graphics frame buffer and other memory regions.

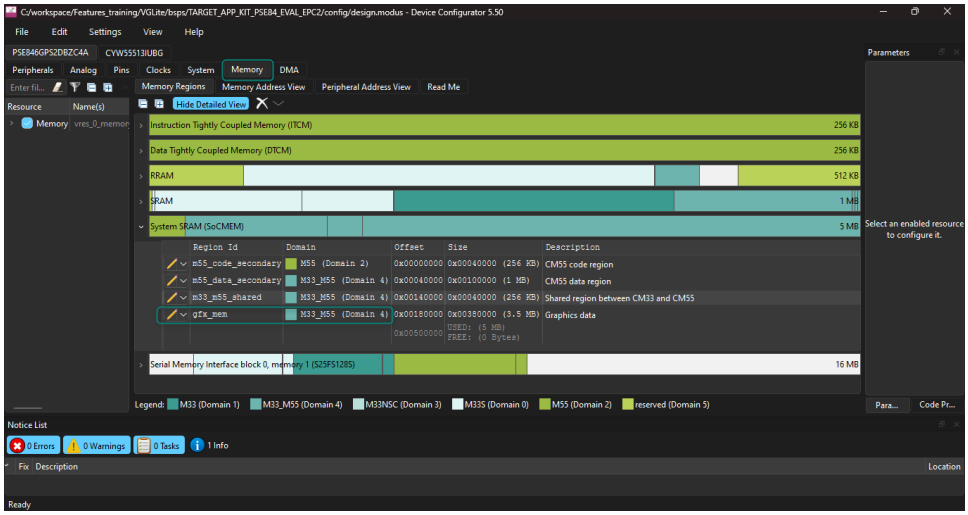


Figure 7 Memory configuration for VGLite example

3.6 Output

The application starts automatically after programming.

You should observe a message in the terminal, followed by a menu, frames-per-second information, and CPU usage:

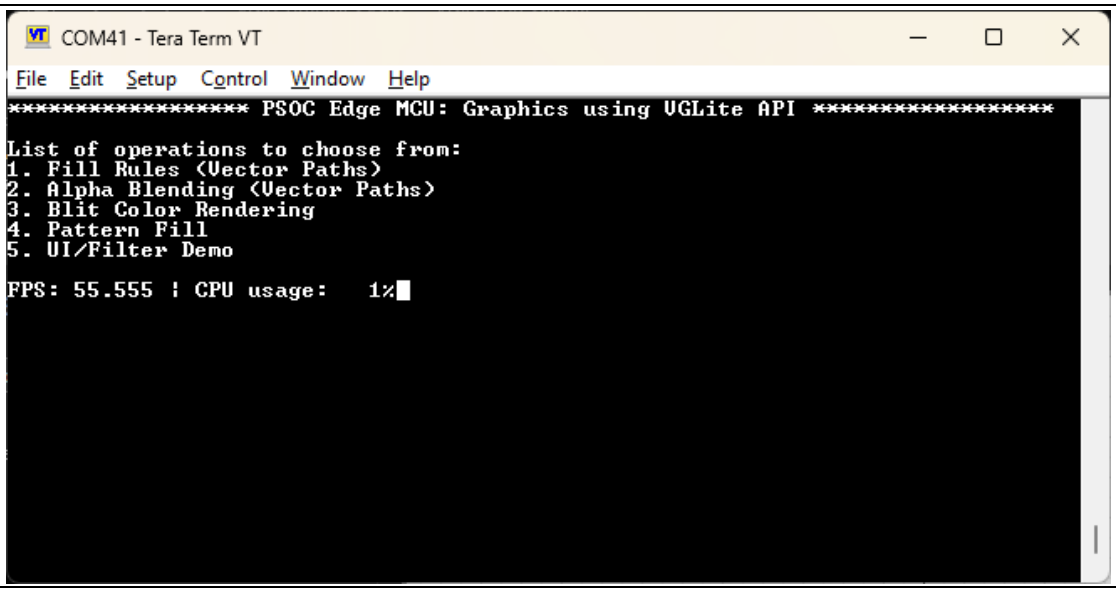


Figure 8 VGLite example output on terminal

You should also observe that the display shows an animated logo.



Figure 9 VGLite example output on display

In addition to the logo display, the example allows you to run different demos:

- Fill rules
  - The display shows two shapes on the left side with the VG_LITE_FILL_EVEN_ODD filling rule and two shapes on the right side with the VG_LITE_FILL_NON_ZERO filling rule.
- Alpha blending
  - The display shows two shapes on the left side with the VG_LITE_BLEND_SRC_OVER blending mode and two shapes on the right side with the VG_LITE_BLEND_MULTIPLY blending mode.
- Blit color rendering
  - The display shows two Infineon logos rectangularly blitted onto a TEAL_COLOR frame.
- Pattern fill
  - The display shows the Infineon logo on four different shapes using the `vg_lite_draw_pattern`.
- UI/filter demo
  - The display shows four different icons and a highlighted rectangle moving over all four icons with VG_LITE_FILTER_LINEAR.

Input the corresponding number in the terminal to run each demo and press **Ctrl+C** to go back to the main menu.

3.7 Modify the application

3.7.1 Render a different image

For our first modification, we will render a different image instead of the Infineon logo.

Hints for the advanced modification

- A sample image *tiger_paths.h* is available in the included source folder
- Adjust image to display properly

Solution

1. Copy *tiger_paths.h* file from the included source files at *~/Graphics_using_VGLite/Render Different* and place it in the proj_cm55 directory.

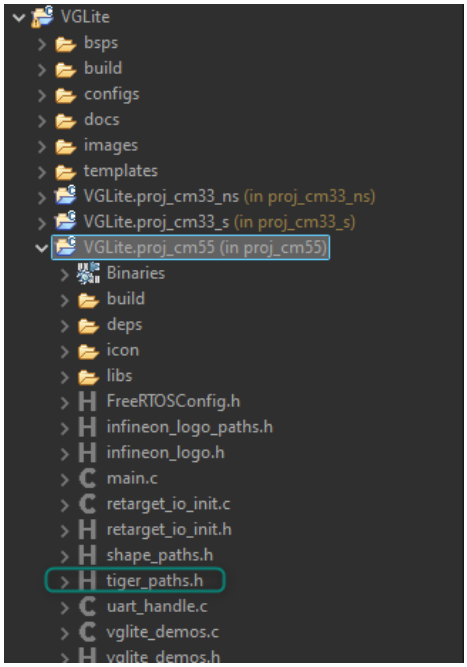


Figure 10 Copy tiger_paths.h

2. Include the *tiger_paths.h* header file in *proj_cm55/vglite_demos.c*, and comment #include "Infineon_logo_paths.h".

Code listing 1 Update path to image

```
#include "icon/vision.h"
#include "icon/wearable.h"
//#include "infineon_logo_paths.h"
#include "tiger_paths.h"
#include "infineon_logo.h"
#include "shape_paths.h"
#include "retarget_io_init.h"
```

3. The *tiger_paths.h* file includes a `color_data` array which conflicts with current definitions in *vglite_demos.c*.
   Comment the following `color_data` definition:

Code listing 2 Comment color_data array

```
/*******************************************************************************
* Global Variables
*******************************************************************************/

bool zoom_out = false;
int scale_count = RESET_VAL;
//uint32_t color_data[] =
//{
//    0xff4018ec, /* path_data0 : blue */
//    0xffb36600, /* path_data1 : red */
//};
vg_lite_buffer_t image_buffer;
```

4. The tiger image has a different size compared to the Infineon logo. Adjust the drawing parameters in *proj_cm55/main.c* as shown below:

Code listing 3 Adjust drawing parameters

```
if (success)
{
    vg_lite_identity(&matrix);

    if (!load_images())
    {
        printf("Failed to load icons\r\n");
        success = false;
```

```
    }
    else
    {
        /* Translate the matrix to the center */
        vg_lite_translate(DISP_W/2 -20  * DISP_W/1024.0f,
        DISP_H /2 -100 * DISP_H /600.0f + 50, &matrix);

        /* Set the default scale of the matrix */
        vg_lite_scale(4, 4, &matrix);
    }
}
```

5. Build and program the application code.
   You should observe the tiger image animated on the display.



Figure 11 VGLite demo output after first modification

3.7.2 Draw polygons

For our second modification, we will explore how to use the *vg_lite_draw* function to draw polygons on the screen.

We will modify the “Fill rules” demo to draw a cross instead of one of the default squares.

Hints for the advanced modification

- The two squares are drawn in `fill_rules_draw` inside *vglite_demos.c*
- The function `vg_lite_draw` draws the polygon using a set of coordinates defined in *shape_paths.h*
- Figure 12 shows the coordinates to draw a rectangle

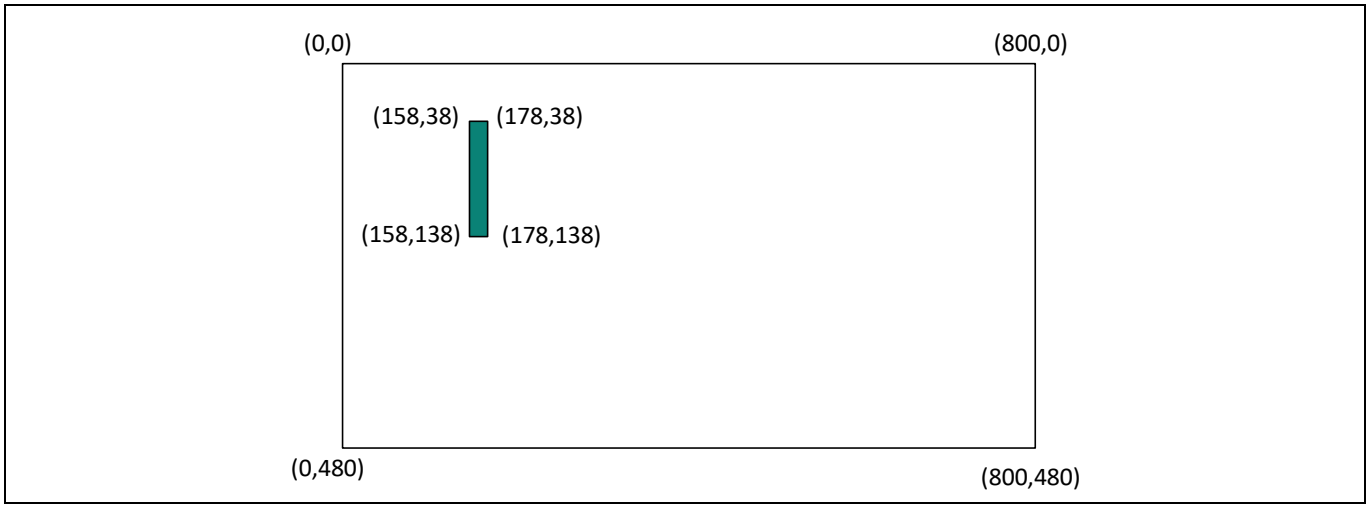


Figure 12 Drawing a rectangle using vg_lite_draw

Solution

1. **Note**: Open *proj_cm55/vglite_demos.c* and observe that the squares are drawn by `vg_lite_draw` and located by `vg_lite_translate`.

```
/* Draw overlapping squares path (bottom-left) */
vg_lite_identity(&matrix);
vg_lite_translate(X_OFFSET_0, SCREEN_HALF_HEIGHT, &matrix);
error = vg_lite_draw(render_target, &overlapping_squares_path, VG_LITE_FILL_EVEN_ODD,
    &matrix, VG_LITE_BLEND_NONE, TEAL_COLOR);
if (error)
{
    printf("vg_lite_draw() returned error %d\r\n", error);
    break;
}// if error

/* Draw concentric circles path (bottom-right) */
vg_lite_identity(&matrix);
vg_lite_translate(SCREEN_HALF_WIDTH, SCREEN_HALF_HEIGHT, &matrix);
error = vg_lite_draw( render_target, &overlapping_squares_path, VG_LITE_FILL_NON_ZERO,
                    &matrix, VG_LITE_BLEND_NONE, TEAL_COLOR );
```

Figure 13 APIs drawing overlapping squares

2. Replace the definition of `overlapping_squares` in *shape_paths.h*.

Code listing 4 Modifying the shape from square to cross

```
int32_t overlapping_squares[] =
{
    /* Cross */
    2, 158, 38,
    4, 178, 38,
    4, 178, 138,
    4, 158, 138,
    4, 158, 38,
    2, 118, 78,
    4, 118, 98,
    4, 218, 98,
```

```
    4, 218, 78,
    4, 118, 78,

/* Second square - Clockwise (shifted) */
    2, 183, 103,
    4, 283, 103,
    4, 283, 203,
    4, 183, 203,
    4, 183, 103,

    0, /* End of path */
};
```

3. Rebuild and reprogram the device.
   Press '1' in the terminal to run the "Fill rules" demo and observe the changes.

Try drawing other shapes or playing with the parameters of `vg_lite_draw` to test other features.

3.8 Conclusion

We successfully created a PSOC™ Edge E84 MCU graphics application using ModusToolbox™. We figured out how to configure the graphics subsystem and the display and touch driver, and we called VGLite API functions to draw using the GPU in the PSOC™ Edge E84 MCU.

4 Code example: Graphics LVGL demo

4.1 Objective

This code example demonstrates displaying graphics on a 4.3” 800x480, or 10.1-inch or 7-inch 1024x600 TFT LCD display using the **Light and Versatile Graphics Library (LVGL)** on PSOC™ Edge E84 MCU. The display shows a music player application, which is listed as one of the standard demos on the LVGL page. The display is connected to the PSOC™ Edge E84 MCU via the **MIPI Display Serial Interface (DSI)**. The code runs in a FreeRTOS environment.

4.2 Description

This code example performs the following major steps to display a GUI created using the LVGL library.

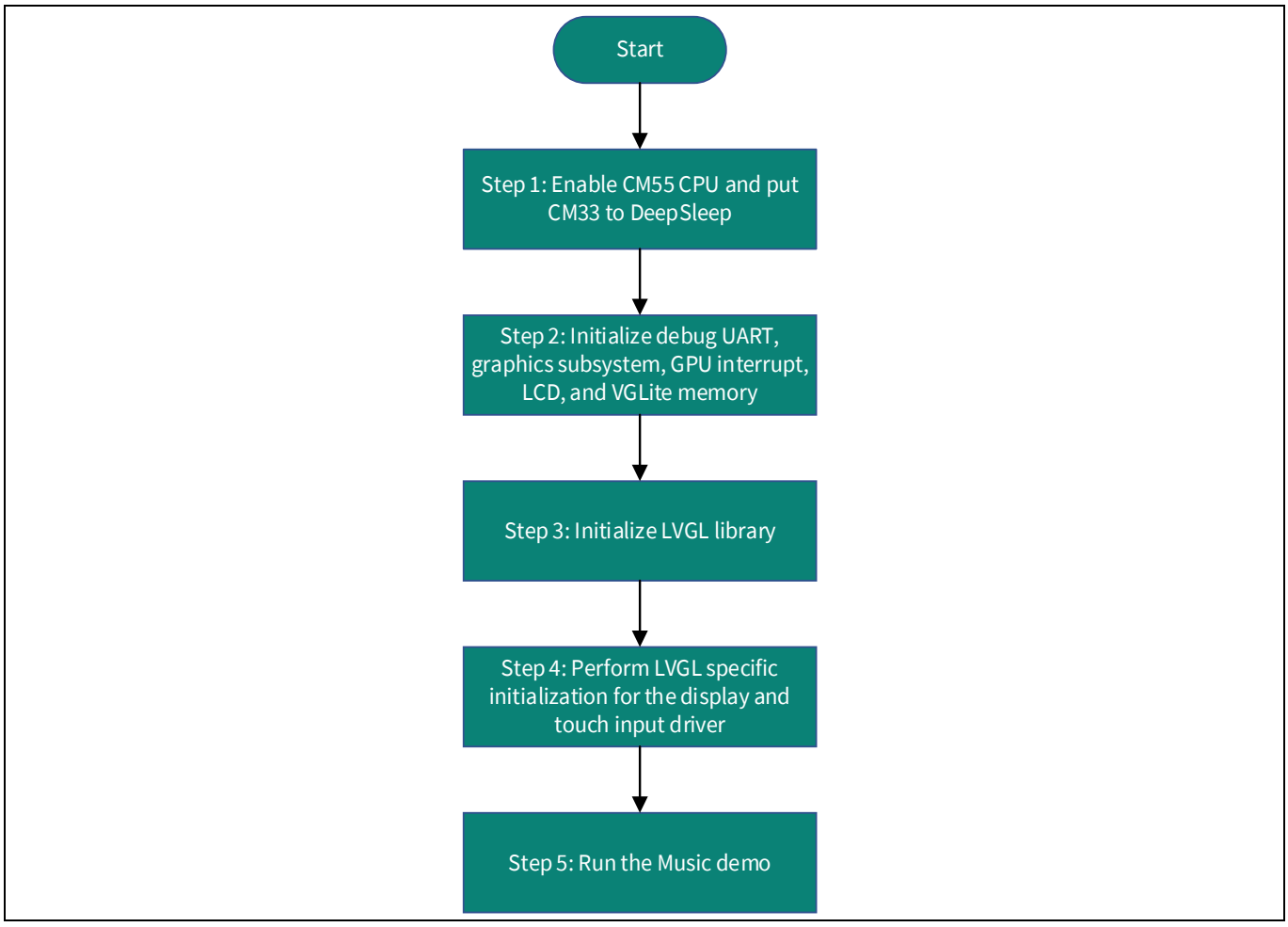


Figure 14 LVGL demo application flow

4.3 Flow chart

The following flow chart explains the complete code flow of this code example.

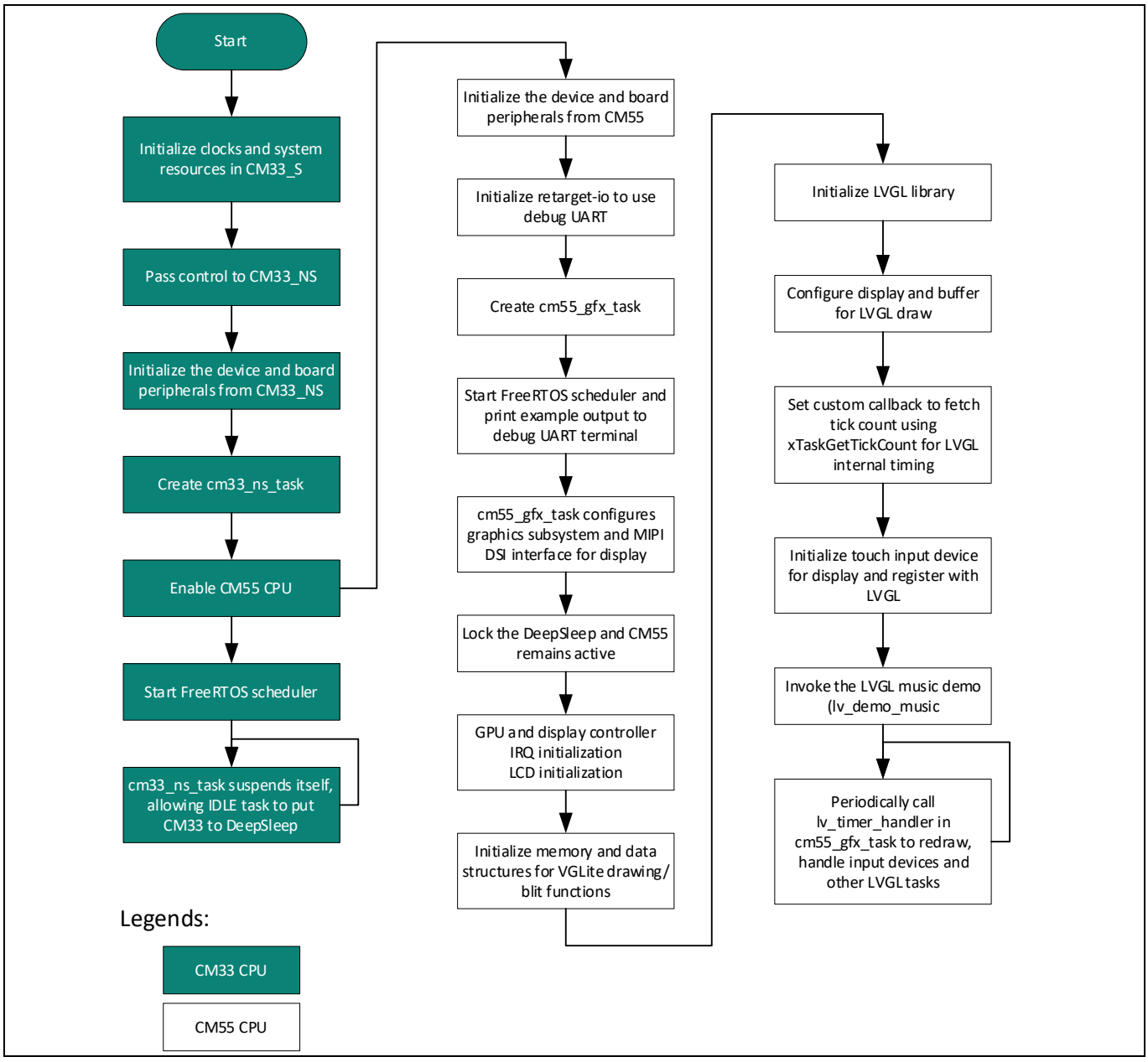


Figure 15 Flow diagram for LVGL example

4.4 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Graphics LVGL Demo** application under the **Graphics** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

Code example: Graphics LVGL demo

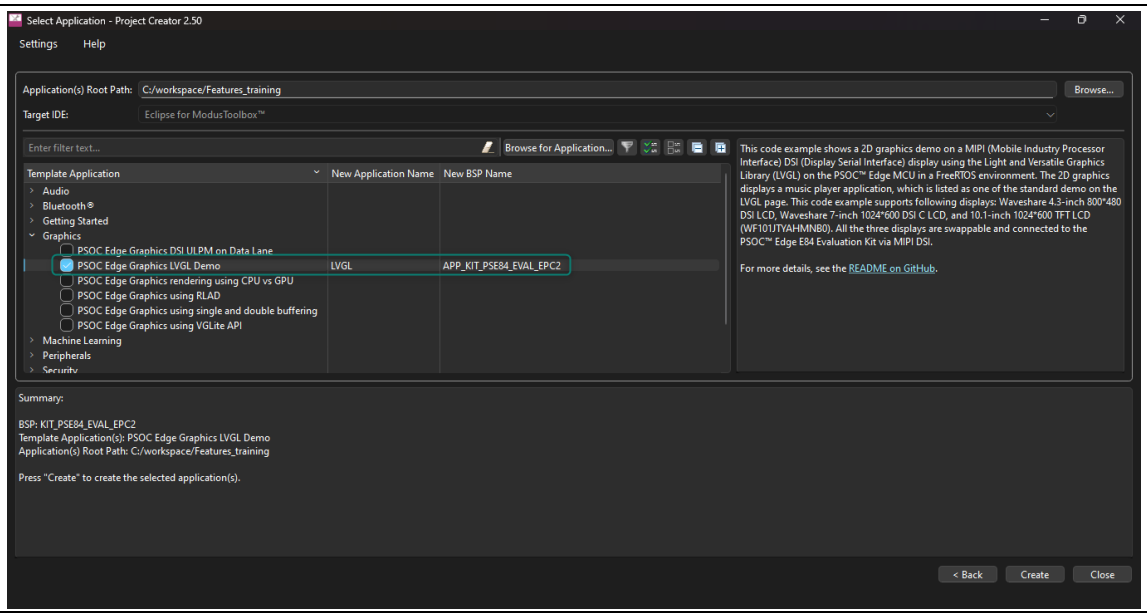


Figure 16 Creating LVGL example

3. Build and program the application.
4. **Note**: No modifications are needed to execute the default code example; however, some important files for this project are highlighted below:

   a) lv_port_disp.c

   - This file provides the implementation of a low-level display device driver for LVGL. It creates a new display device for LVGL with the given resolution, sets up the frame buffer for the display, initializes a custom callback to fetch the tick count, and provides functions for refreshing or updating the display.
   - The file also contains implementations of functions required by LVGL to get information about the display and how to draw pixels on the screen, such as `disp_flush()`. The **`disp_flush()`** function is called by LVGL whenever a new frame is ready to be displayed, and it transfers the contents of the graphics frame buffer to the display controller.
   - This file is typically customized to work with specific display hardware. It may require modification of certain constants or register settings to properly configure the display. The specific implementation may also depend on the type of display being used, such as a TFT or OLED display.

   b) lv_port_indev.c

   - This file provides the implementation of a low-level input device driver for LVGL. It initializes the touch driver to the corresponding display panel using the I2C interface and handles user input such as touch or button presses. It provides functions that map input events to specific actions or GUI elements in your LVGL application.
   - The file also contains implementations of functions required by LVGL to get information from the input device, such as `touchpad_read()`. The **`touchpad_read()`** function is called by LVGL to read input events from the input device and translate them into LVGL event types, such as LV_EVENT_PRESSED or LV_EVENT_RELEASED.
   - This file is typically customized to work with a specific input device. It may require modification of certain constants or register settings to properly configure the input device. The specific implementation may also depend on the type of input device being used, such as a touch screen or physical buttons.

   c) lv_conf.h

- This is a configuration file for the LVGL graphics library. It allows you to customize various aspects of the library, such as the display driver, memory allocation, theme, font, and more.
- The file provides a list of preprocessor directives that can be used to enable or disable certain features of the library. By modifying the directives in this file, you can tailor LVGL to suit your specific needs and hardware platform. It is typically included in your project along with the LVGL library files.

d) lv_draw_vg_lite.c, lv_draw_vg_lite_img.c

- These files provide implementation of LVGL's image related operations ported to the VGLite library to use GPU aided rendering.

e) lv_vg_lite_utils.c

- This file provides an implementation for LVGL's utility APIs ported to the VGLite library.

f) proj_cm55/main.c

- This file defines the **`cm55_gfx_task`** which initializes the graphics subsystem and configures the GPU and display controller interrupts. After that, the file initializes the display panel driver for the selected display.
- Once the panel is initialized, the required amount of memory is allocated for VGLite draw/blit functions to be consumed by the LVGL library.
- The **`lv_init()`** function is used to initialize LVGL and set up the essential components required for LVGL to work correctly.
- Low-level display and touch devices for LVGL are initialized using **`lv_port_disp_init()`** and **`lv_port_indev_init()`** functions respectively.
- The LVGL demo music player is displayed by calling the LVGL demo widget API **`lv_demo_music()`**.

5. **Note**: We can also observe the libraries included in this example by opening the **Library Manager** in the **Quick Panel**.

   Note: Observe the added libraries for LVGL and 4.3" display. Note that the Library Manager can also be used to add support for this display and LVGL to other applications.

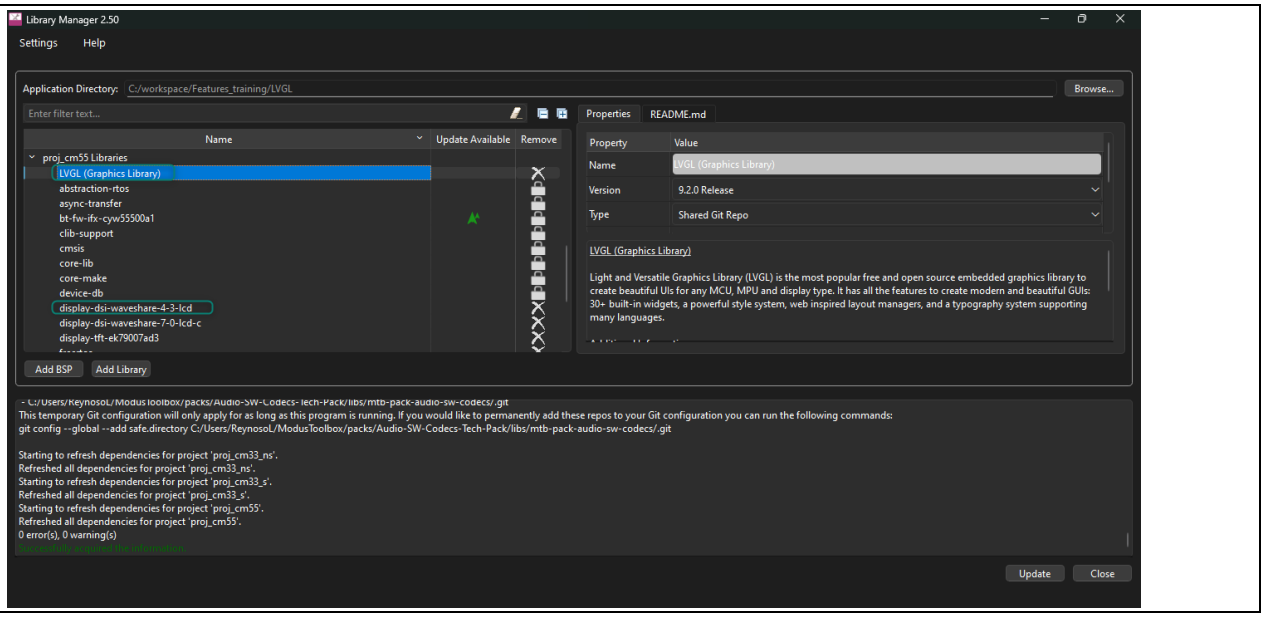


Figure 17 Library manager for LVGL example

4.5 Output

After programing is successful, open the serial terminal and set the serial port parameters to **8N1** and **115200** baud. Reset the EVK and observe that the terminal displays the header.

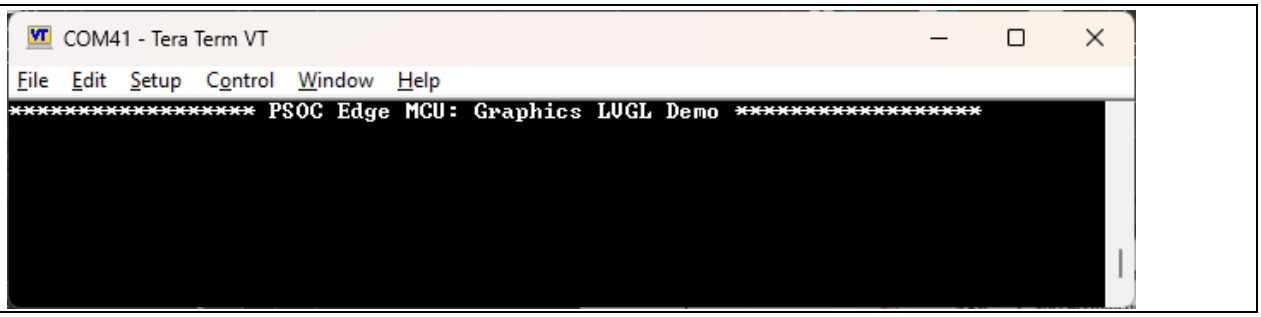


Figure 18 Terminal output for LVGL example

Observe that the LCD displays a music player demo application as shown below. You can use the touch screen to perform various actions such as playing or pausing a track, changing to the next or previous track, and viewing the playlist. (Music is not played, only graphics actions are demonstrated).

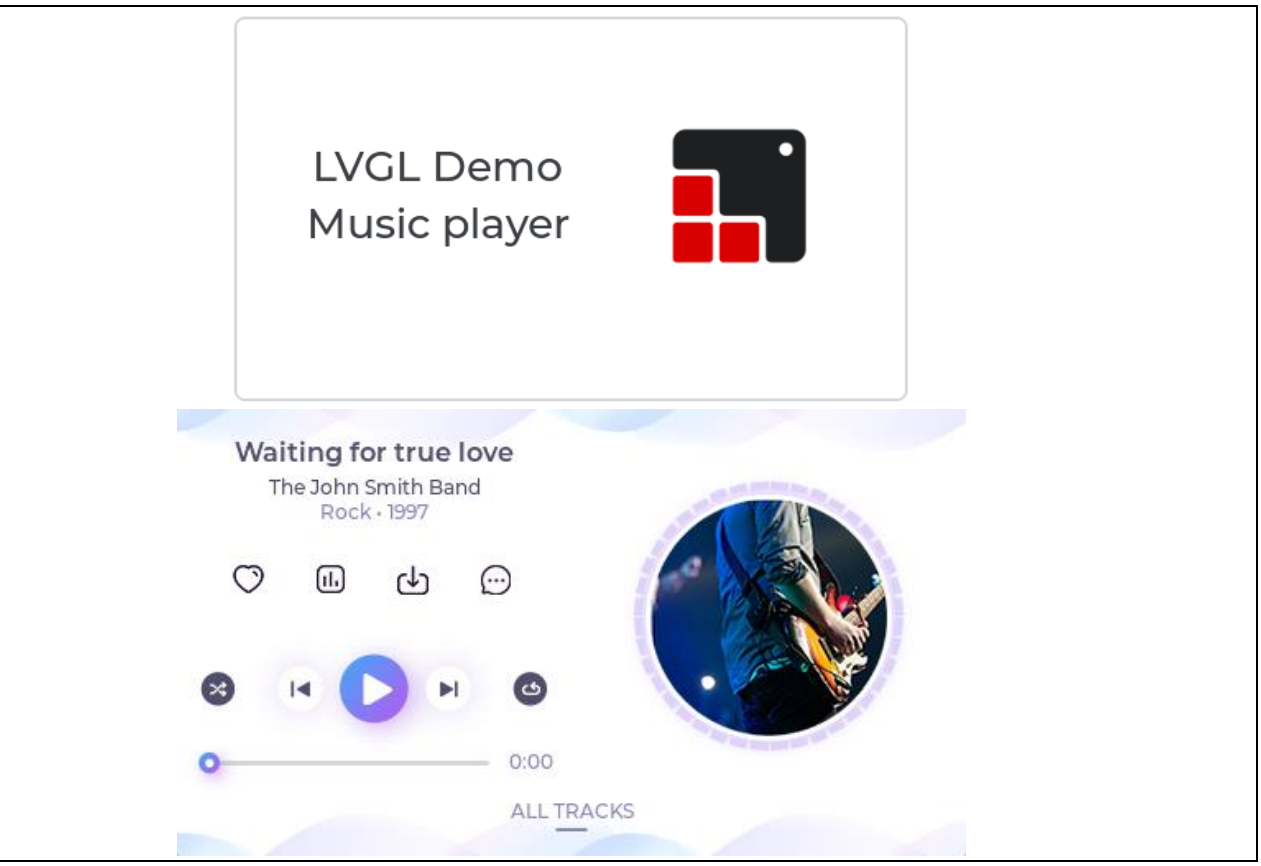


Figure 19 Display output for LVGL example

4.6 Modify the application

4.6.1 Pre-built LVGL demos

For our first modification, try testing other pre-built LVGL demos, and enable CPU usage indicators.

Hints for the advanced modification

- LVGL demos and CPU usage indicators can be enabled in *lv_conf.h*
- Make sure the application calls the correct `lv_demo` function

Solution

1. To switch to other available LVGL demos, enable the corresponding **LV_USE_DEMO_<demo_name>** macro in *proj_cm55\lv_conf.h*.
   Note: In the example below, we will enable the Widgets demo.

Code listing 5 Enable widgets demo in lv_conf.h

```
/*====================
 * DEMO USAGE
 ====================*/

/*Show some widget. It might be required to increase `LV_MEM_SIZE` */
#define LV_USE_DEMO_WIDGETS 1

/*Demonstrate the usage of encoder and keyboard*/
#define LV_USE_DEMO_KEYPAD_AND_ENCODER 0

/*Benchmark your system*/
#define LV_USE_DEMO_BENCHMARK 0

/*Render test for each primitives. Requires at least 480x272 display*/
#define LV_USE_DEMO_RENDER 0

/*Stress test for LVGL*/
#define LV_USE_DEMO_STRESS 0

/*Music player demo*/
#define LV_USE_DEMO_MUSIC 0
#if LV_USE_DEMO_MUSIC
```

2. Let's enable the system monitor component by enabling **LV_USE_SYSMON** in the same *lv_conf.h* file. This will allow us to observe CPU usage metrics.

Code example: Graphics LVGL demo

Code listing 6 Enable system monitor in lv_conf.h

```
/*==================
 * OTHERS
 *==================*/

/*1: Enable API to take snapshot for object*/
#define LV_USE_SNAPSHOT 0

/*1: Enable system monitor component*/
#define LV_USE_SYSMON   1
#if LV_USE_SYSMON
```

3. Call the corresponding **lv_demo_<demo_name>()** API in place of lv_demo_music in *proj_cm55/main.c*.

Code listing 7 Call the widgets demo in main.c

```
if (VG_LITE_SUCCESS == vglite_status)
{
    /* Initialize LVGL library */
    lv_init();
    lv_port_disp_init();
    lv_port_indev_init();

    /* Run the Music demo */
    //lv_demo_music();
    /* Run the Widgets demo */
    lv_demo_widgets();
}
```

4. Rebuild and reprogram the application.
5. You should observe the widgets demo together with CPU metrics in the bottom left corner.

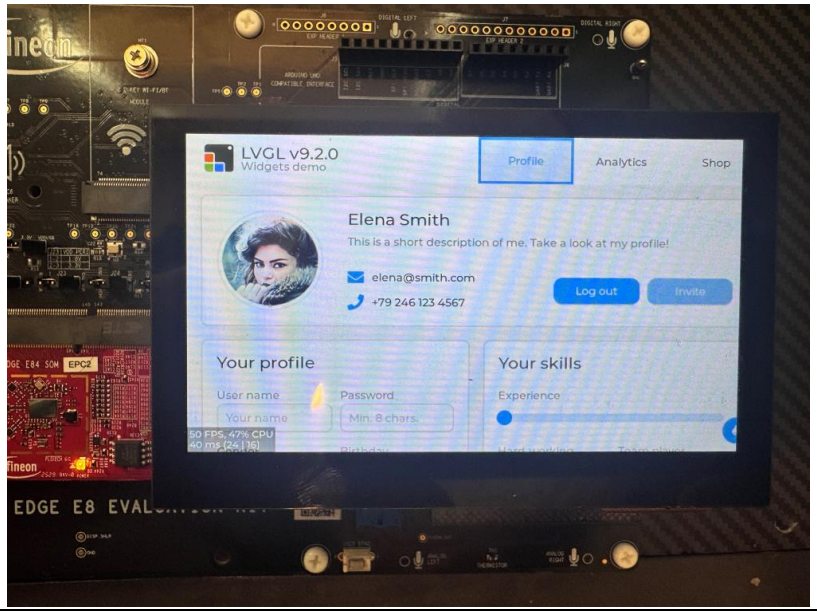


Figure 20 Display after enabling LVGL widgets demo

4.6.2 Custom UI using EEZ Studio

LVGL is a widely used graphics library supported by the open-source community and many third parties. In this exercise, we will explore how to generate a custom UI using the EEZ Studio tool.

Hints for the advanced modification

- Use EEZ Studio to generate a new UI
- Add the generated UI files to the cm55 project
- Call the corresponding functions to initialize the custom UI and update it periodically

Solution

1. If you haven't already installed EEZ Studio, install it now following instructions in the Tools for graphics labs section.
2. Open EEZ Studio and click on **Examples**, select the **Change Screen** example under the LVGL category, select a **name**, add a **location**, and click in **Edit Project**.

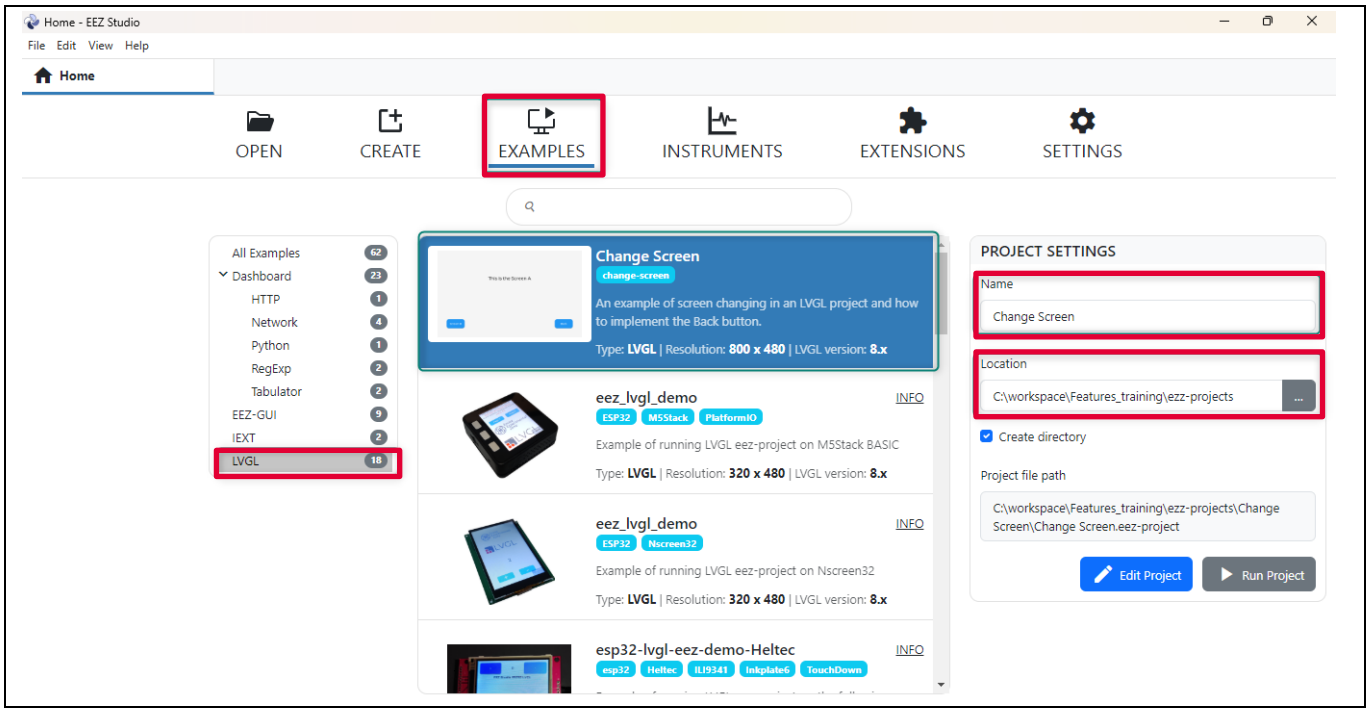


Figure 21 Creating example in EEZ Studio

3. Click the **Settings** button.

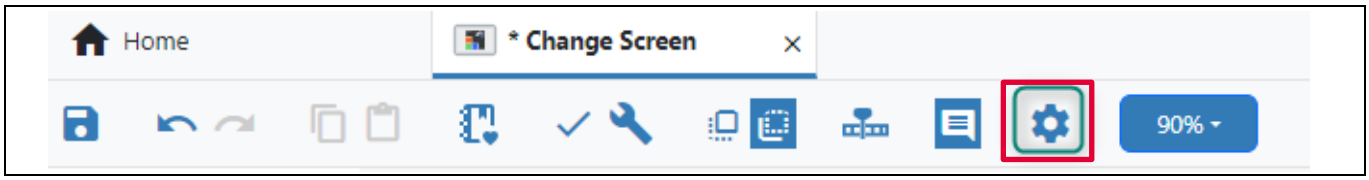


Figure 22 Open settings in EEZ Studio

4. In the **General** tab, confirm that the **LVGL version** is 9.x since ModusToolbox™ 3.6 uses LVGL 9.2.0, and that the **display resolution** is 800x480, since we are using the 4.3” display included with the PSOC™ Edge E84 kit.

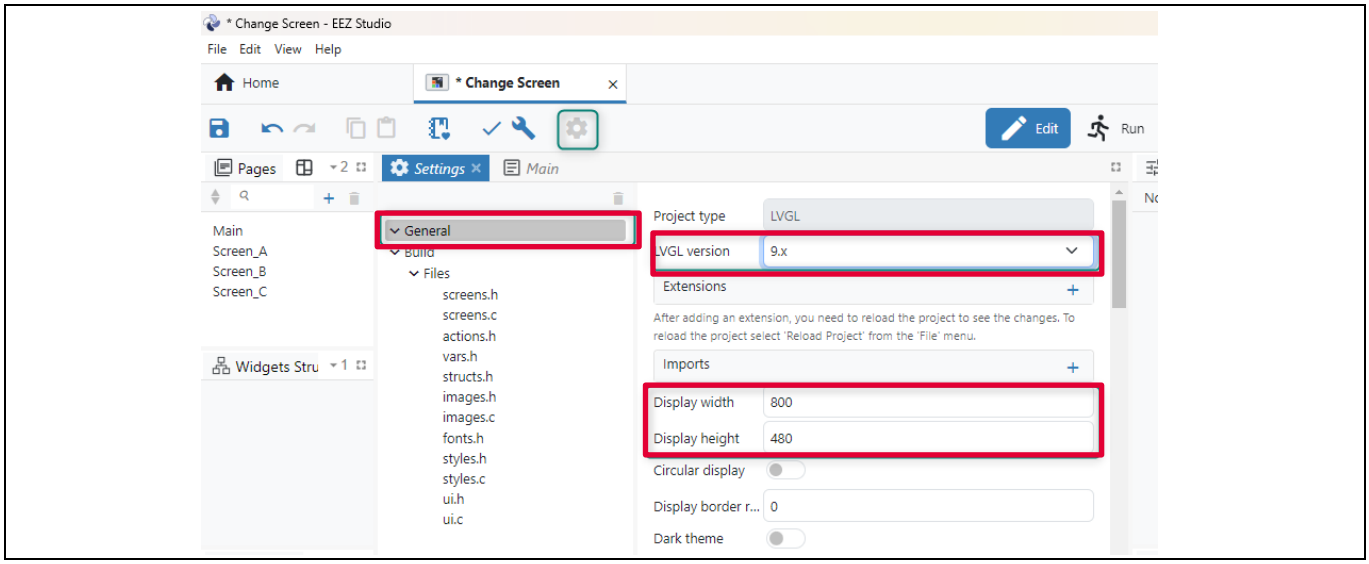


Figure 23 General settings in EEZ Studio

5. Now go to the **Build** options and modify the **LVGL include** setting as “lvgl.h”, since our project is configuring a direct path to LVGL inside mtb_shared; and observe that the **Destination folder** is set to “*src\ui*” since this is where the output files will be generated.

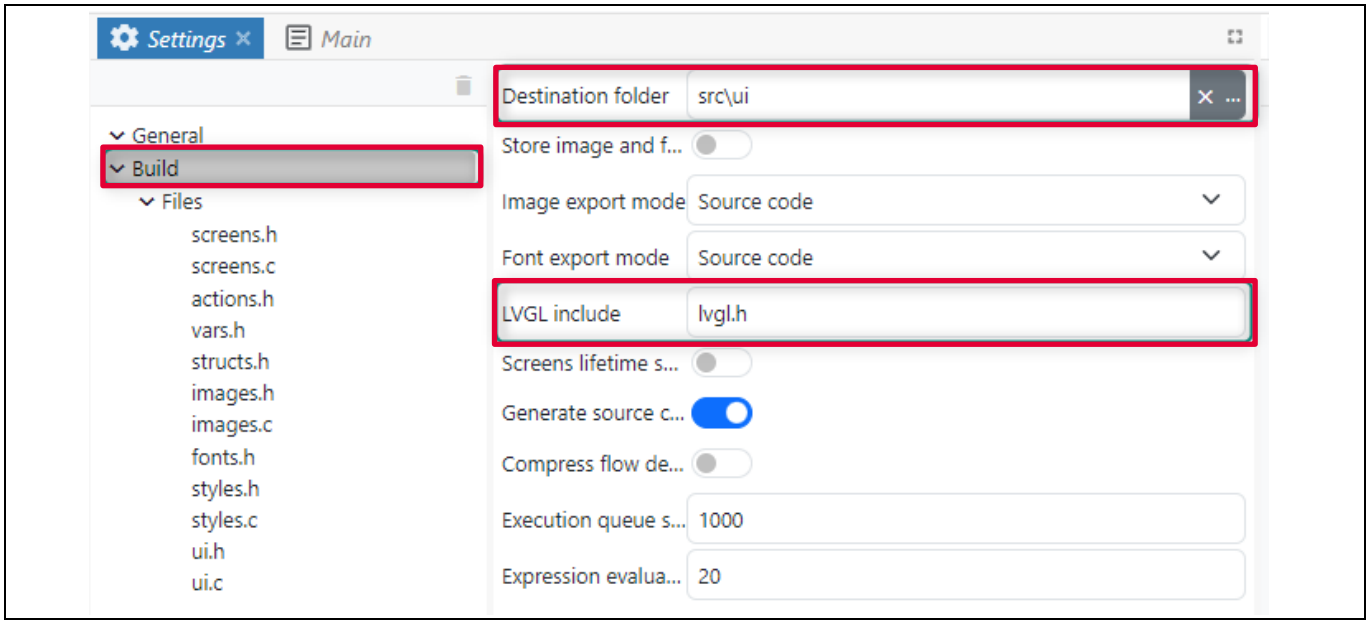


Figure 24 Build settings

6. Let's modify the default GUI. Make sure you are on the **Main page**, then open the **Bitmaps** panel on the right side, and click the **plus (+) icon** to add a bitmap.

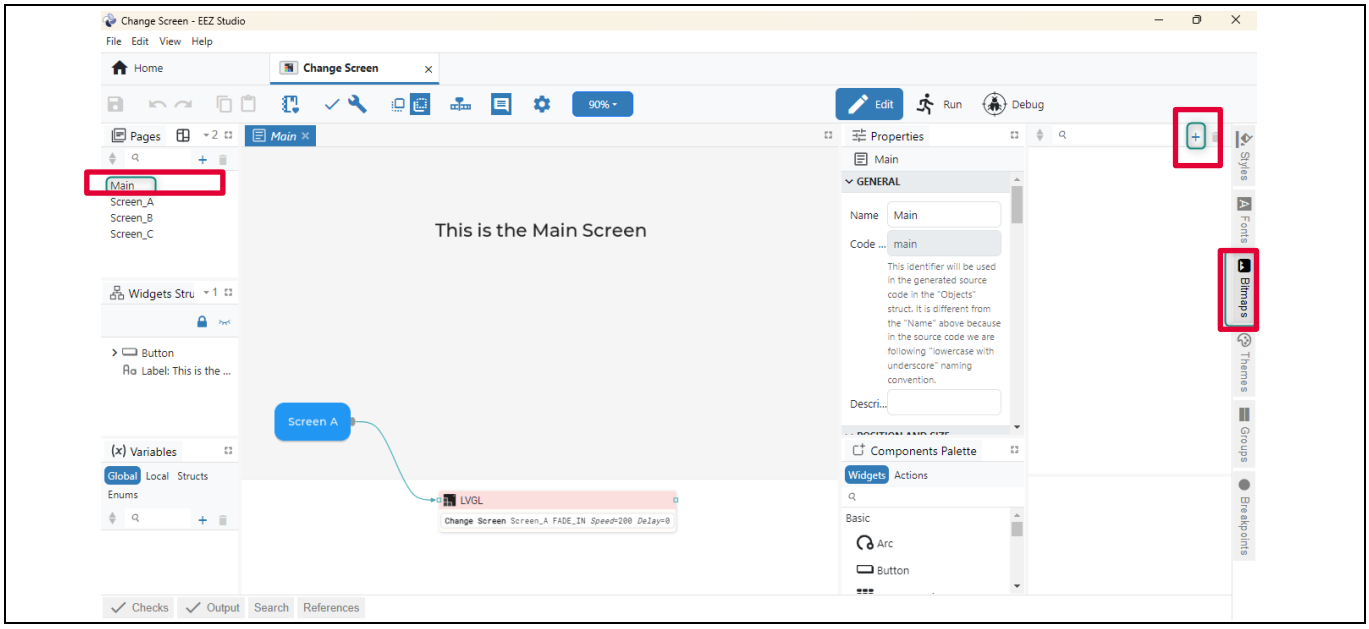


Figure 25 Adding a bitmap in EEZ Studio UI

7. Click the image browse button, navigate to the lab source folder, and select the image located at *<lab_source_folder>\Graphics_LVGL_Demo\Custom UI using EEZStudio\PSE84.png*.
   Add a **Name**, and select **Color format** as RGB565, then click **OK**.
   Note: Feel free to add an image of your choice.

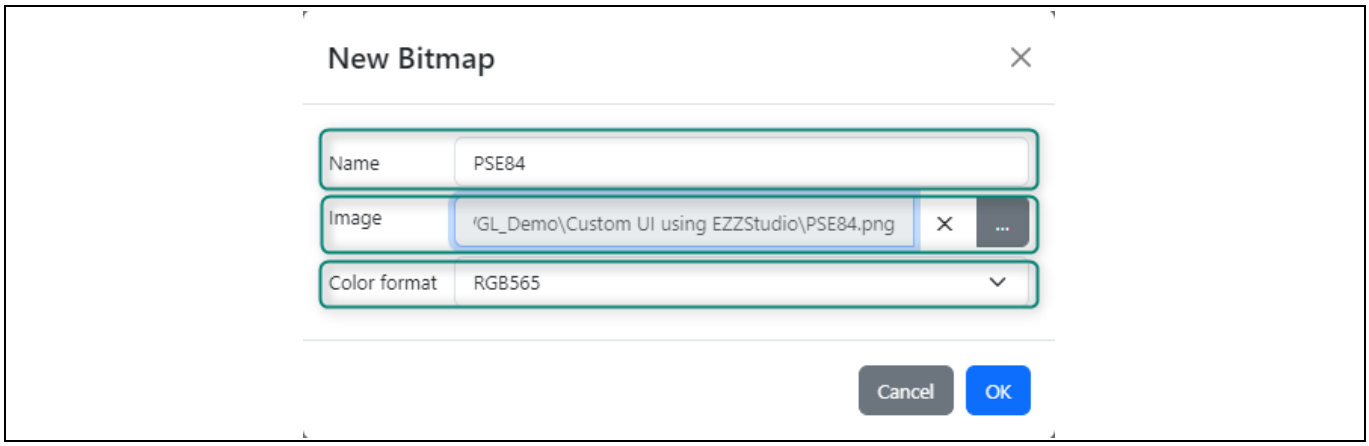


Figure 26 Adding image as bitmap

8. Drag-and-drop an **Image widget** from the **Components Palette** into the **Main Screen**.
   Then, go to the **Properties** and select the **Image** corresponding to the bitmap added in the previous step.

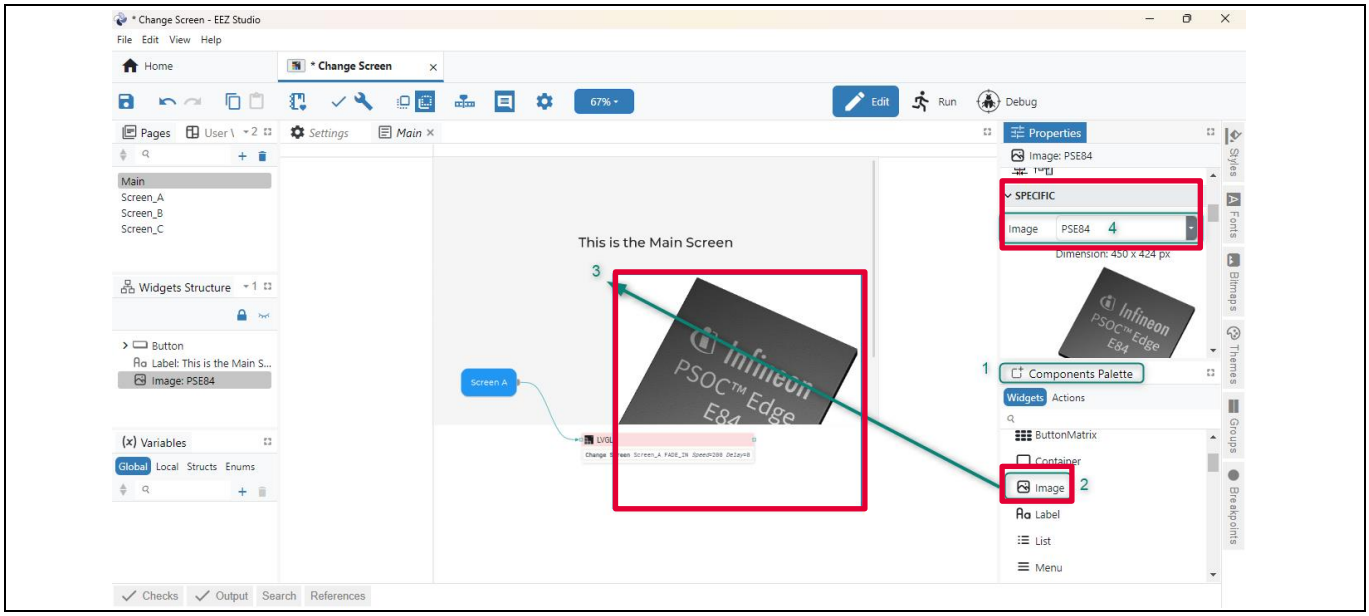


Figure 27 Add an image to the main screen

9. Adjust the label and image so that it fits properly in the page.
10. We will test one of the widgets on **Screen A**.
    Go to the **Screen A** page and drag and drop an **Arc widget** from the **Components Palette**.
    Note: Feel free to try other widgets.

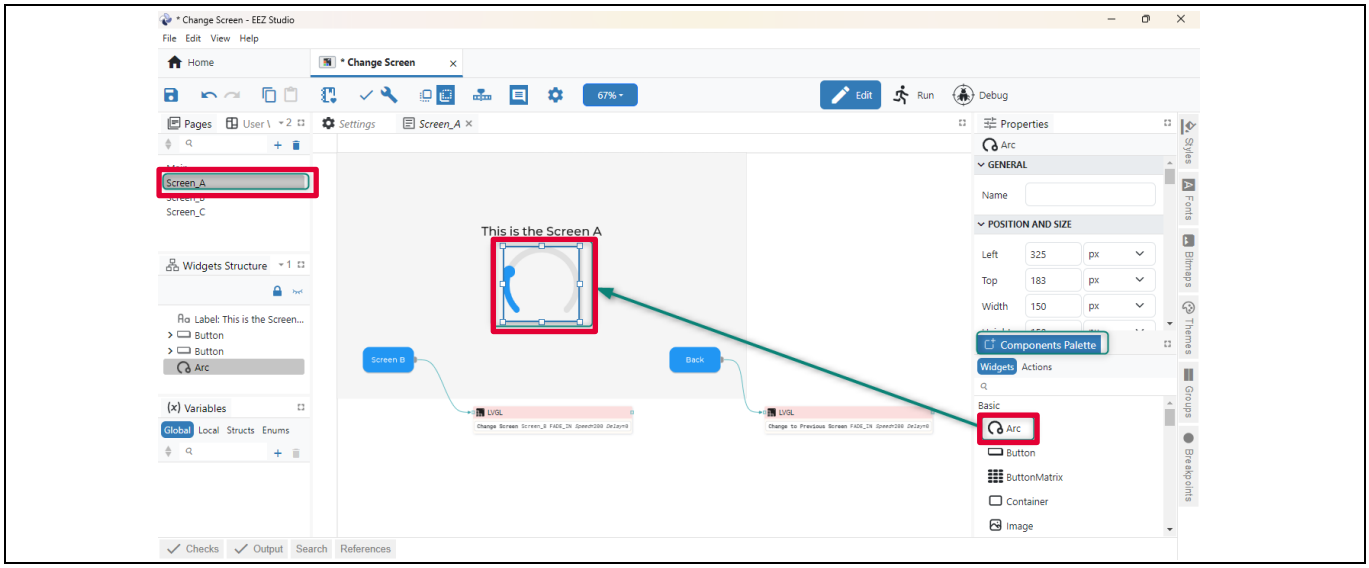


Figure 28 Adding an Arc widget to Screen A

Save the project, click the **Check** button to validate the project, and then click on the **Build** button to generate the output files.

Note: EEZ Studio will display any missing system requirements. The known requirements are listed in the Tools for graphics labs section.

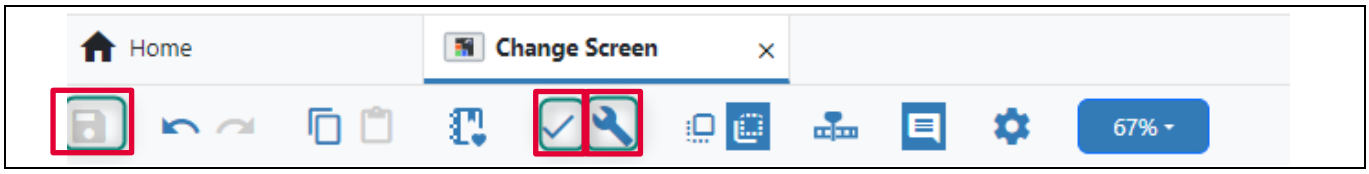


Figure 29 Build the EEZ Studio project

11. Copy the generated **UI folder** from *<EEZ_Project_path>/src/ui* to the *proj_cm55*.
    Tip: Right click on the project tap to find and copy project path.

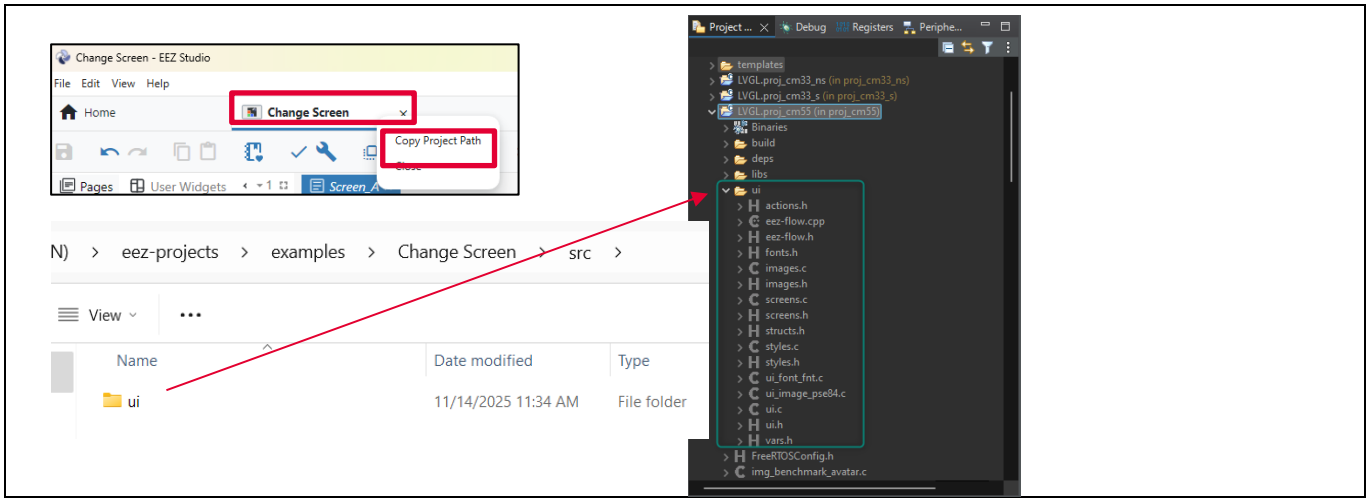


Figure 30 Add generated UI folder to proj_cm55

12. Open *proj_cm55/main.c* and make the following modifications to initialize the new GUI and to call a periodic tick function to handle UI events.

Code listing 8 Modifications to main.c to initialize custom UI

```
#include " display_i2c_config.h"
#include "ui.h"
```

```
if (VG_LITE_SUCCESS == vglite_status)
{
    /* Initialize LVGL library */
    lv_init();
    lv_port_disp_init();
    lv_port_indev_init();

    /* Run the custom UI */
    ui_init();
}
```

Code listing 9 Modifications to main.c to call periodic UI tick

```
for (;;)
{
    /* LVGL's timer handler function, to be called periodically to handle
    * LVGL tasks.
    */
    time_till_next = lv_timer_handler();
    vTaskDelay(pdMS_TO_TICKS(time_till_next));
    ui_tick(); /* Call UI tick to handle events */
}
```

13. Rebuild and program the example.
14. You should observe the new GUI running on your PSOC™ Edge EVK. Try the different screens and/or perform more customizations.

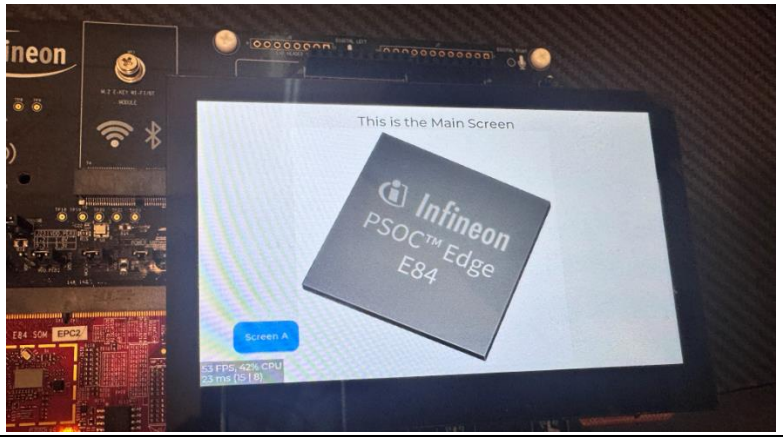


Figure 31 Display after running custom UI

4.7 Conclusion

We successfully created a PSOC™ Edge E84 MCU graphics application using ModusToolbox™, figured out how to configure the graphics subsystem, and utilized the LVGL graphics library to develop a full-fledged UI application on the PSOC™ Edge E84 MCU platform.

5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisites section.

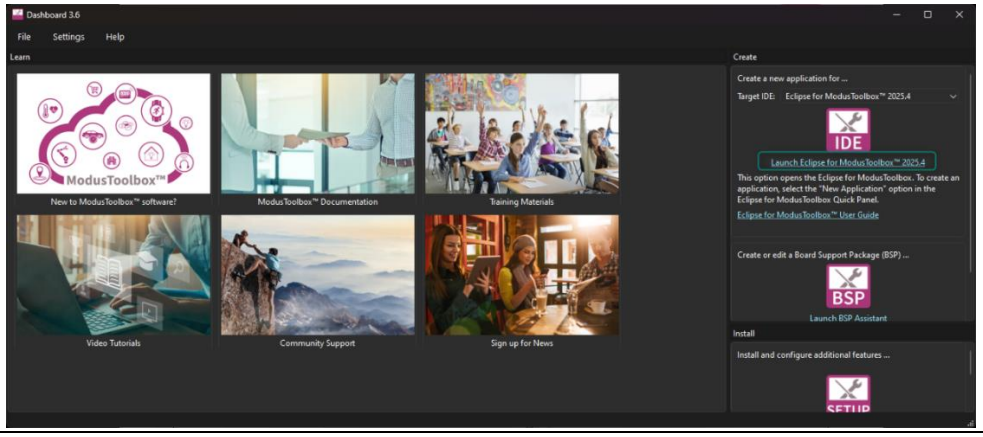


Figure 32 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

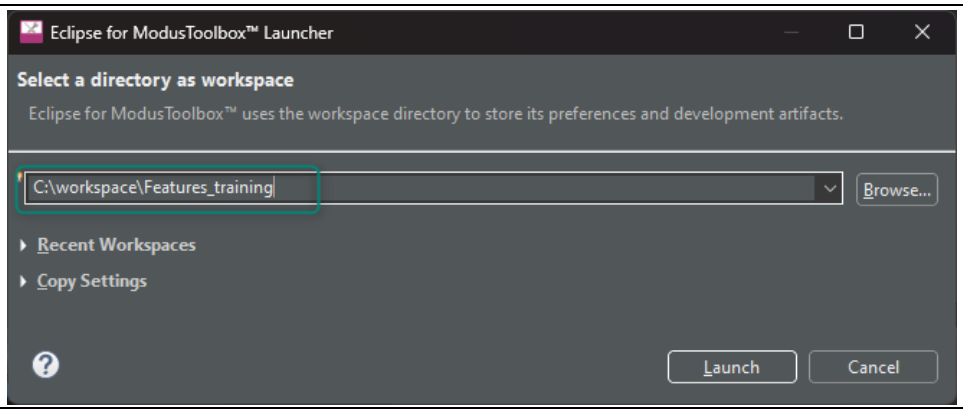


Figure 33 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

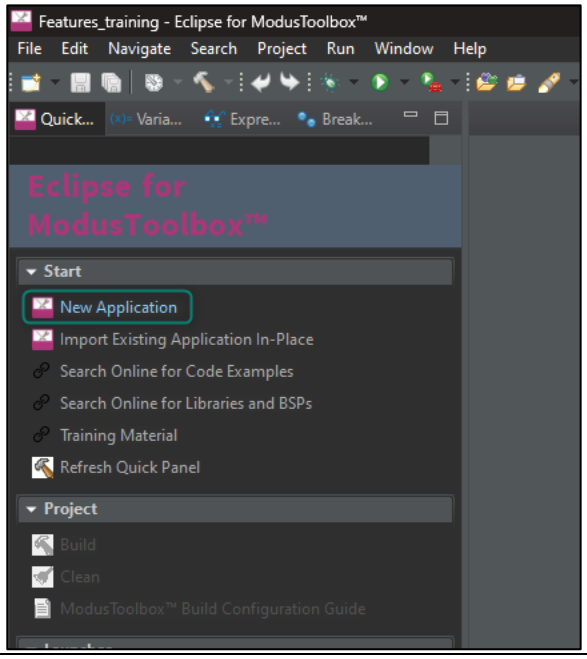


Figure 34 Create new application

4. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP). Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.

   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*

   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

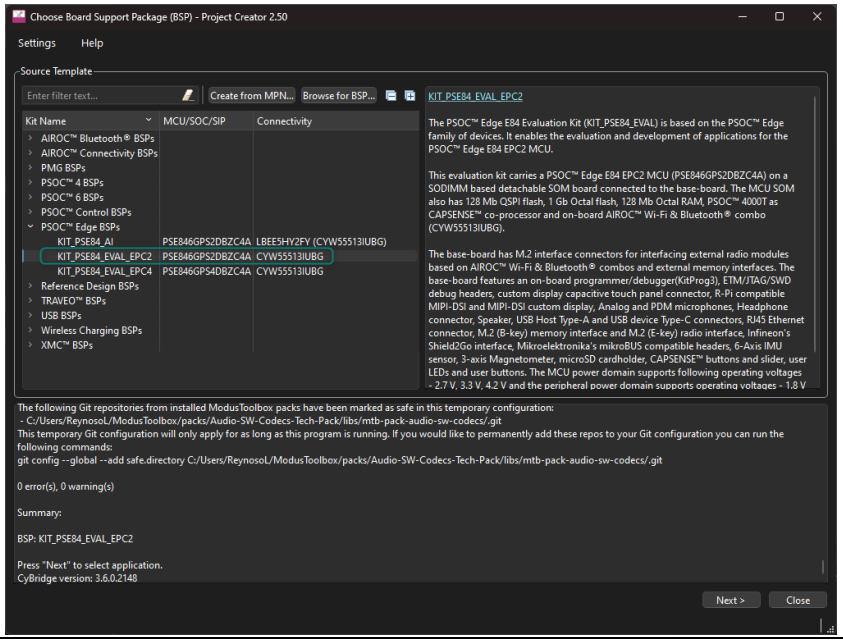


Figure 35 Selecting BSP in Project Creator

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.

   Figure 36 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

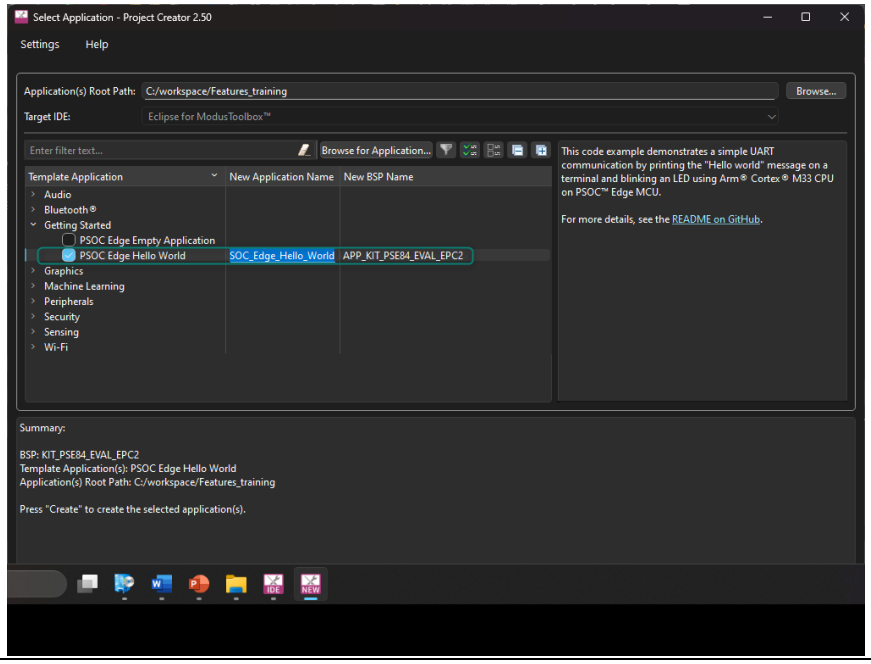


Figure 36 Selecting application

6. A new project is created, and with all the files visible in the **Project Explorer** window.

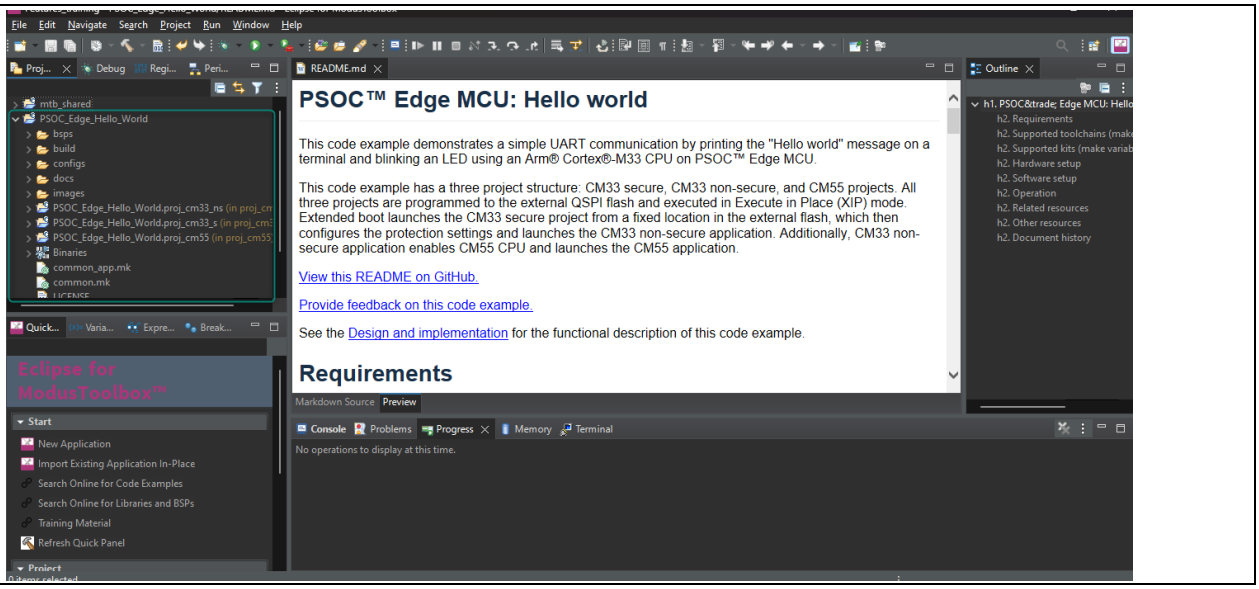


Figure 37 New application created

6 Appendix B: KIT_PSE84_EVAL details

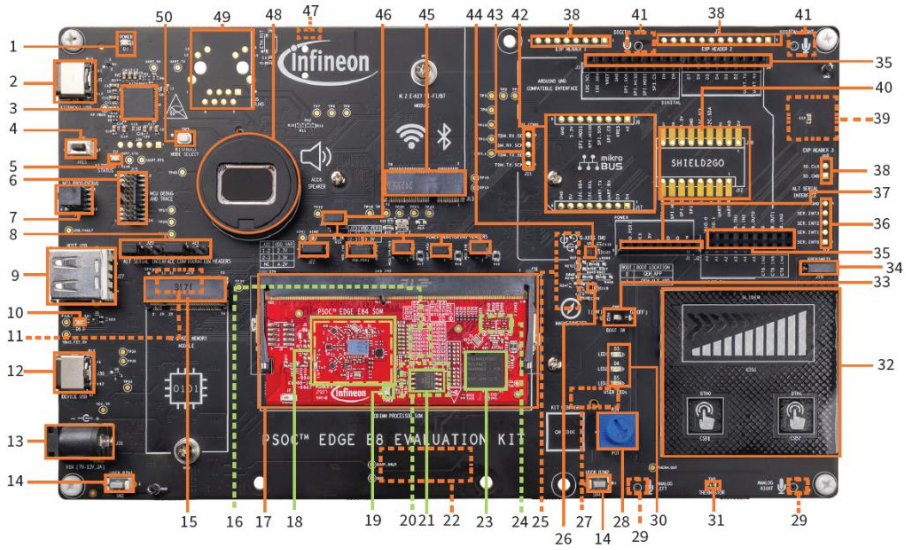


Figure 38 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2025-01-15	Initial release.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-15

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com